

# Proyecto Tickets TD

José Ignacio Calatayud Cuenca, Marc Donato Merí, Manuel Giner Flores, Marc Navarro Montava, Edua

2025-03-27

## Algoritmo del proyecto.

## Introducción.

Este proyecto tiene como objetivo desarrollar un algoritmo en el lenguaje de programación R capaz de realizar un análisis exhaustivo de tickets de compra en formato PDF, específicamente de los obtenidos en establecimientos de Mercadona. A través de este proceso, se busca extraer la información relevante de cada ticket y convertirla en un formato estructurado, como lo es el data frame, que facilita su análisis y manipulación.

La importancia de este proyecto radica en la posibilidad de automatizar el proceso de recopilación de datos de tickets en formato PDF, que a menudo son difíciles de manejar debido a su estructura no estandarizada. El uso de R para esta tarea ofrece una gran flexibilidad y potencia a la hora de trabajar con datos y aplicar técnicas de análisis estadístico y visualización.

El enfoque del proyecto se basa en la creación de un algoritmo eficiente que sea capaz de leer, interpretar y extraer información clave de los tickets, tales como: fecha de la compra, productos adquiridos, cantidades, precios y, especialmente, el impacto de factores como el IVA en el precio final de los productos. Este análisis podría proporcionar valiosas perspectivas sobre cómo ha afectado la evolución del IVA a los precios de productos específicos, lo que permitiría realizar comparativas a lo largo del tiempo.

Mediante el código en R hemos llegado a extraer la información de los tickets y hemos obtenido dos data frame tales que:

## Datos del data frame general del ticket

```
knitr::kable(head(tickets_global,5),  
              caption = "Ejemplo del data frame con la información general referente al ticket",  
              booktabs = TRUE)
```

Table 1: Ejemplo del data frame con la información general referente al ticket

| archivo                           | ubicacion                  | codigo_producto | posicion      | telefono  | fecha      | hora  | op | factura_simplificada | importe | total      | AID        | AUTARC   | tipos_iva      |
|-----------------------------------|----------------------------|-----------------|---------------|-----------|------------|-------|----|----------------------|---------|------------|------------|----------|----------------|
| 20231125 Mer-cadona 37,76 Ôé¼.pdf | CTRA. GAR-RUCHA A VERA S/N | 04630           | GARRUCHA      | 966516065 | 11/12/2023 | 11:33 | 25 | 2916-010-520925      | 37,76   | 0727000700 | 0000000000 | 0312848  | 3010%, 21%     |
| 20231201 Mer-cadona 86,70.pdf     | AVDA. VA-LEN-CIA 41        | 03830           | MURODE AL-COY | 966516065 | 12/12/2023 | 11:49 | 32 | 2916-011-451305      | 86,70   | 0035864270 | 0000000000 | 07410843 | 3000%, 21%, 0% |
| 20231205 Mer-cadona 58,00.pdf     | AVDA. VA-LEN-CIA 41        | 03830           | MURODE AL-COY | 966516065 | 12/12/2023 | 11:49 | 22 | 2916-011-452354      | 58,00   | 0035864270 | 0000000000 | 04800533 | 3000%, 5%, 0%  |
| 20231213 Mer-cadona 29,34.pdf     | C/ ALI-CANTE 83            | 03801           | ALCOY         | 966516065 | 12/12/2023 | 11:36 | 79 | 2916-017-211832      | 29,34   | 0349030880 | 0000000000 | 02487743 | 3000%, 21%, 0% |
| 20231215 Mer-cadona 3,20.pdf      | AVDA. VA-LEN-CIA 41        | 03830           | MURODE AL-COY | 966516065 | 12/12/2023 | 11:36 | 72 | 2916-010-731651      | 3,20    | 0035864270 | 0000000000 | 09189243 | 3000%, 601     |

Los datos generales referentes al ticket se muestran en la Tabla @ref(tab:tabla-autos).

## Datos del data frame de productos del ticket

```
knitr::kable(head(productos_global,5),
  caption = "Ejemplo del data frame con la información de los productos del ticket",
  booktabs = TRUE)
```

Table 2: Ejemplo del data frame con la información de los productos del ticket

| Descripcion | P.Unit | Peso | Importe | Tipo_peso | factura_simplificada |
|-------------|--------|------|---------|-----------|----------------------|
| DONACIÓN    | 1      | NA   | NA      | NA        | 2916-010-520925      |
| DONACIÓN    | 1      | NA   | NA      | NA        | 2916-010-520925      |
| DONACIÓN    | 1      | NA   | NA      | NA        | 2916-010-520925      |
| DONACIÓN    | 1      | NA   | NA      | NA        | 2916-010-520925      |
| DONACIÓN    | 1      | NA   | NA      | NA        | 2916-010-520925      |

Los datos específicos referentes a los productos del ticket se muestran en la Tabla @ref(tab:tabla-autos).

De este modo, será posible responder a preguntas clave relacionadas con la evolución de precios, las preferencias de consumo y los patrones de compra:

Empezamos resolviendo aquellas planteadas como comunes.

- ¿Cuáles son los 5 productos, de los vendidos por unidades, con más ventas? ¿Cuántas unidades de cada uno se han vendido?

```
productos_ud_masvendidos <- productos_global %>%
  # Filtramos aquellos que no tienen peso, buscamos únicamente los que van por unidades.
  filter(is.na(Peso)) %>%
  # No queremos contar a las bolsas de plástico y Parking por obvias razones.
  filter(!Descripcion %in% c("BOLSA PLASTICO", "PARKING")) %>%
  # Le quitamos los espacios en blanco al principio y al final de cada cadena de Descripcion.
  mutate(Descripcion = str_trim(Descripcion)) %>%
  # En una nueva columna llamada como n_apariciones contamos cuántas veces aparece cada descripción y
  count(Descripcion, sort = TRUE, name = "n_apariciones") %>%
  # Mostramos las 5 primeras.
  head(5)

productos_ud_masvendidos
```

```
##           Descripcion n_apariciones
## 1  ATUN CLARO OLIVA          62
## 2  LECHE DESNAT. CALCIO       49
## 3  QUESO LONCHAS CABRA        49
## 4      YOGUR COCO            40
## 5    PAN SEMILLAS            38
```

- Si consideramos la categoría de frutas y verduras, ¿cuáles son los 5 productos más vendidos? ¿Cuántos kilos se han vendido de cada uno de estos productos?

```
# Obtenemos los nombres de los 5 productos más vendidos.
productos_fv_masvendidos_nombres <- productos_global %>%
  # Filtran aquellas que tengan peso y que sean verduras o frutas.
  filter(!is.na(Peso), Tipo_peso == "Verdura y Frutas") %>%
  # Contamos las veces que se repiten.
  count(Descripcion, sort = TRUE) %>%
  # Tomamos únicamente las cinco primeras filas.
  slice_head(n = 5) %>%
  # Extraemos la columna como un vector de caracteres.
  pull(Descripcion)

productos_fv_masvendidos_nombres
```

```
## [1] "PLATANO"      "BANANA"      "PEPINO"      "BERENJENA"   "KIWI VERDE"
```

```
# Calculamos la suma del peso de los productos que más se repiten.
productos_fv_masvendidos <-
  productos_global %>%
  # Filtramos para conservar aquellas que su peso era diferente de NA.
  filter(!is.na(Peso)) %>%
  # Nos aseguramos que este dentro de los 5 más repetidos.
  filter(Descripcion %in% productos_fv_masvendidos_nombres) %>%
  # Agrupamos por descripción.
  group_by(Descripcion) %>%
  # Hacemos la suma del peso total.
```

```
summarise(Peso_total_kg = sum(Peso, na.rm = TRUE)) %>%
# Ordenamos de forma descendente.
arrange(desc(Peso_total_kg))
```

```
productos_fv_masvendidos
```

```
## # A tibble: 5 x 2
##   Descripcion Peso_total_kg
##   <chr>          <dbl>
## 1 PLATANO        62.9
## 2 BANANA         28.1
## 3 PEPINO         19.6
## 4 BERENJENA      14.3
## 5 KIWI VERDE     6.31
```

- Si consideramos la categoría de pescado, ¿cuáles son los 5 productos más vendidos? ¿Cuántos kilos se han vendido de cada uno de estos productos?

```
# Nos aseguramos antes de que Peso sea numérico:
productos_global <- productos_global %>%
  mutate(Peso = str_replace_all(Peso, ",", ".")) %>% as.numeric()
```

```
# Los 5 pescados más vendidos, excluyendo otras categorías.
productos_p_masvendidos_nombres <-
  productos_global %>%
  # Filtramos para que tenga peso y que sea pescado.
  filter(
    !is.na(Peso),
    Tipo_peso == "Pescado") %>%
  # Contamos las veces que se repiten.
  count(Descripcion, sort = TRUE) %>%
  # Tomamos únicamente las cinco primeras filas.
  slice_head(n = 5) %>%
  # Extraemos la columna como un vector de caracteres.
  pull(Descripcion)
```

```
productos_p_masvendidos_nombres
```

```
## [1] "BACALADILLA"          "SEPIA FRESCA"          "SEPIA LONJA"
## [4] "LENGUADO FRESC EUROP" "PESCADO"
```

```
# Sumamos su peso total, asegurándonos de volver a filtrar pescado.
productos_p_masvendidos <- productos_global %>%
  # Filtramos para conservar aquellas que su peso era diferente de NA y que su tipo sea pescado. Además
  filter(
    !is.na(Peso),
    Tipo_peso == "Pescado",
    Descripcion %in% productos_p_masvendidos_nombres) %>%
  # Agrupamos por descripción.
  group_by(Descripcion) %>%
  # Hacemos la suma del peso total.
  summarise(Peso_total_kg = sum(Peso, na.rm = TRUE), .groups = "drop") %>%
```

```
# Ordenamos de forma descendiente.
arrange(desc(Peso_total_kg))
```

```
productos_p_masvendidos
```

```
## # A tibble: 5 x 2
##   Descripcion      Peso_total_kg
##   <chr>          <dbl>
## 1 BACALADILLA      7.37
## 2 SEPIA LONJA      3.58
## 3 SEPIA FRESCA     2.64
## 4 PESCADO          1.51
## 5 LENGUADO FRESC EUROPE 1.25
```

- Muestra mediante un gráfico de líneas cómo ha variado el precio por kilo de las bananas y los plátanos en los tickets disponibles, a lo largo del tiempo.

```
# Filtramos solo bananas y plátanos, y calculamos el precio por kilo.
bananas_platanos <- productos_global %>%
  # Estandarizamos a mayúsculas para captar variantes.
  mutate(Descripcion = str_to_upper(Descripcion)) %>%
  filter(
    str_detect(Descripcion, "\\b(PLÁTANO|PLATANO|BANANA)\\b") &
    !str_detect(Descripcion, "MANZANA")
  ) %>%
  # Calculamos el precio por kg.
  mutate(precio_kg = Importe / Peso) %>%
  # Unimos con la fecha del ticket.
  left_join(
    tickets_global %>%
      select(factura_simplificada, fecha),
    by = "factura_simplificada"
  ) %>%
  # Convierto fecha a Date
  mutate(fecha = dmy(fecha)) %>%
  filter(!is.na(fecha))
```

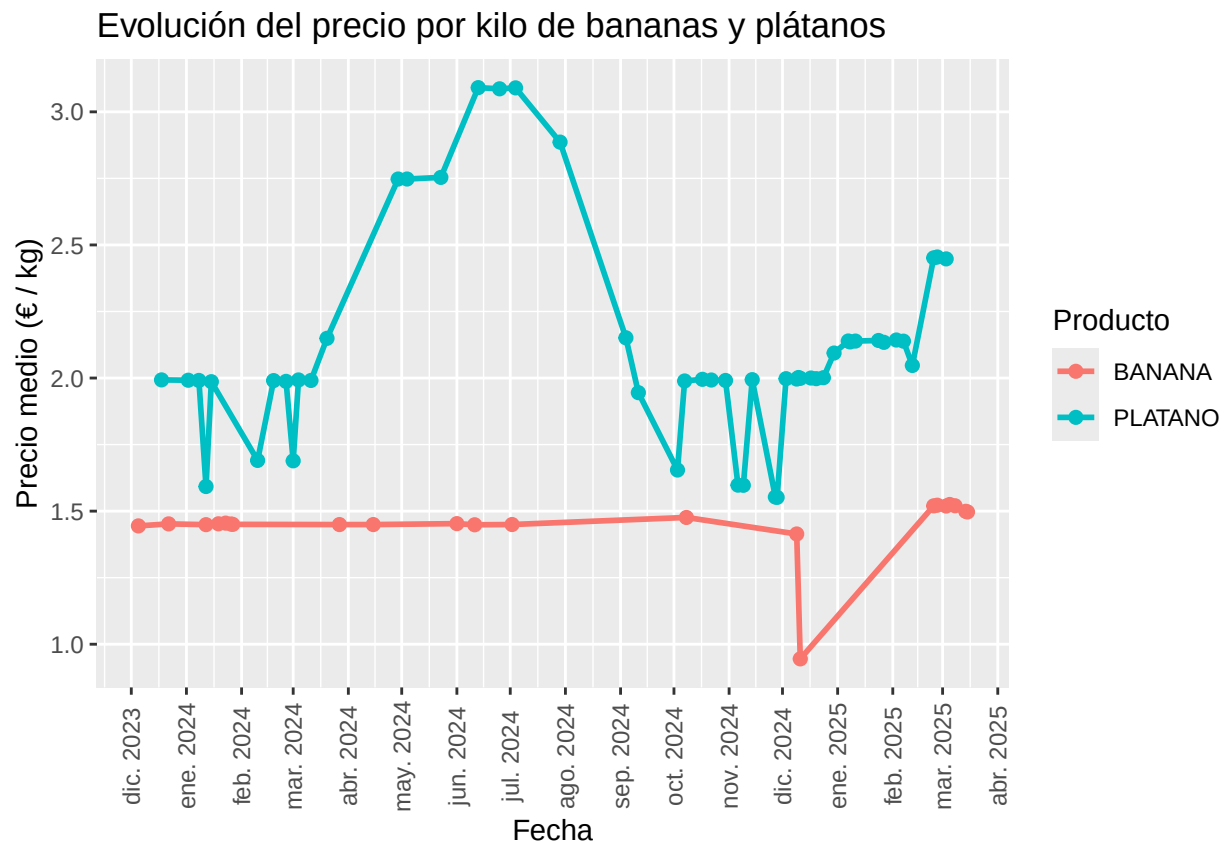
```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many-to-many relationship.
## i Row 2 of 'x' matches multiple rows in 'y'.
## i Row 6 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many"' to silence this warning.
```

```
# Agrupamos por fecha y producto, calculamos precio medio por día.
serie_temporal <- bananas_platanos %>%
  group_by(fecha, Descripcion) %>%
  summarise(precio_medio_kg = mean(precio_kg, na.rm = TRUE), .groups = "drop") %>%
  arrange(fecha)

# Realizamos el gráfico de líneas.
ggplot(serie_temporal, aes(x = fecha, y = precio_medio_kg, color = Descripcion)) +
  geom_line(size = 1) +
```

```
geom_point(size = 2) +
scale_x_date(date_breaks = "1 month", date_labels = "%b %Y") +
labs(
  title = "Evolución del precio por kilo de bananas y plátanos",
  x = "Fecha",
  y = "Precio medio (€ / kg)",
  color = "Producto"
) +
theme(
  axis.text.x = element_text(angle = 90, vjust = 0.5)
)
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



- ¿Cuál es la procedencia de los tickets? ¿Qué ciudad o pueblo tiene un mayor número de tickets?

```
tickets_global %>%
  # Contamos las veces que se repiten las localidades ordenándolas de mayor a menor.
  count(localidad, sort = TRUE) %>%
  # Mostramos las 10 primeras.
  head(10)
```

|       | localidad         | n   |
|-------|-------------------|-----|
| ## 1  | VALENCIA          | 128 |
| ## 2  | ALBORAIA/ALBORAYA | 50  |
| ## 3  | BURJASSOT         | 26  |
| ## 4  | MURO DE ALCOY     | 24  |
| ## 5  | ALCOI/ALCOY       | 22  |
| ## 6  | BUÑOL             | 12  |
| ## 7  | BENIJOFAR         | 9   |
| ## 8  | ALGINET           | 5   |
| ## 9  | BENIFAIO          | 4   |
| ## 10 | GARRUCHA          | 4   |

Como podemos observar Valencia es la ciudad con más tickets.

- Muestra mediante un diagrama el número de tickets recogidos cada día de la semana. ¿Si tuvieses que cerrar un día entre semana, qué día lo harías?

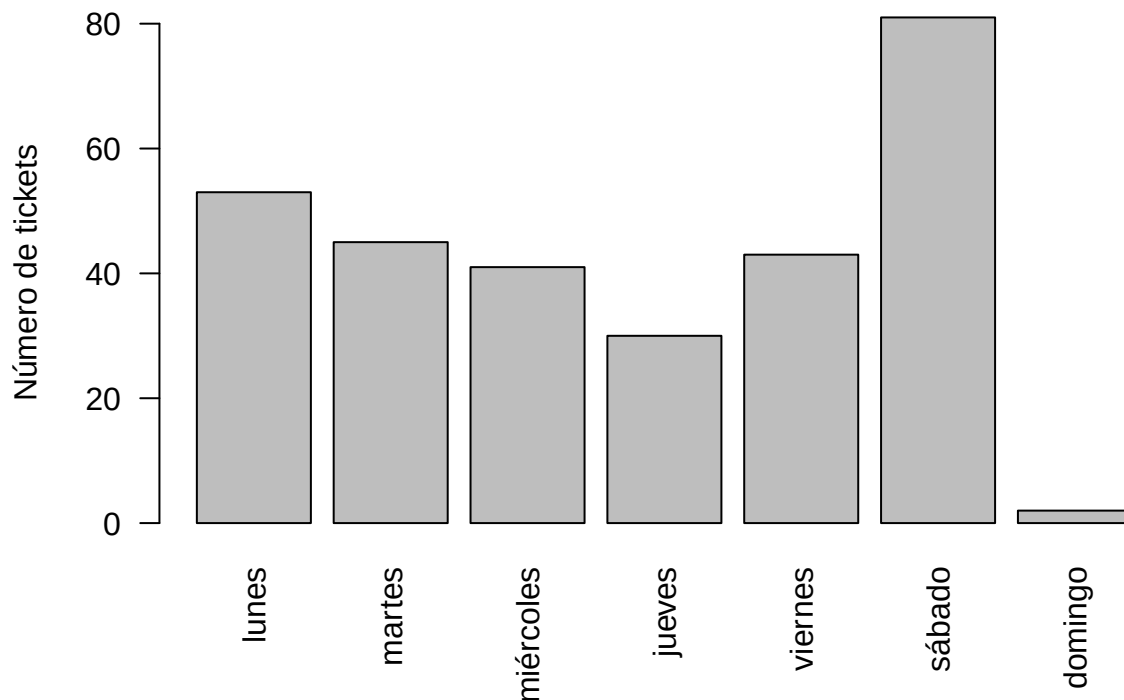
```
# No aseguramos de que la fecha es Date.
tickets_global_2 <- tickets_global %>%
  mutate(fecha = dmy(fecha))

# Hacemos la tabla de recuentos.
recuento_dias <- table( weekdays(tickets_global_2$fecha, abbreviate = FALSE) )

# Reordenamos para mostrar de lunes a domingo.
niveles <- c("lunes","martes","miércoles","jueves","viernes","sábado","domingo")
recuento_dias <- recuento_dias[ tolower(names(recuento_dias)) %in% niveles ]
recuento_dias <- recuento_dias[niveles]

# Hacemos un barplot.
barplot(
  recuento_dias,
  las = 2,                # etiquetas verticales
  main = "Tickets por día de la semana",
  ylab = "Número de tickets"
)
```

## Tickets por día de la semana



Aquellas planteadas por nuestro grupo.

- ¿Cuánto ha subido el precio de la carne en el último año?

```
# Filtrar productos cárnicos (ejemplos representativos)
productos_carne <- c("FILETE DE TERNERA", "SOLOMILLO DE CERDO", "POLLO ENTERO", "LOMO DE CERDO",
                    "CHULETAS DE CERDO", "CARNE PICADA", "HAMBURGUESAS DE TERNERA")

# Obtener datos de productos cárnicos
carne_data <- productos_global %>%
  filter(str_detect(toupper(Descripcion), paste(productos_carne, collapse = "|"))) %>%
  left_join(tickets_global, by = "factura_simplificada") %>%
  mutate(
    fecha = as.Date(fecha, format = "%d/%m/%Y"),
    P.Unit = as.numeric(P.Unit),
    mes = floor_date(fecha, "month")
  ) %>%
  filter(fecha >= Sys.Date() - years(1)) # Último año

# Calcular variación de precios
precio_promedio <- carne_data %>%
  group_by(mes, Producto = Descripcion) %>%
  summarise(Precio_Promedio = mean(P.Unit, na.rm = TRUE), .groups = "drop") %>%
  arrange(Producto, mes)

# Calcular porcentaje de cambio
```



```

variacion <- precio_promedio %>%
  group_by(Producto) %>%
  summarise(
    Precio_Inicial = first(Precio_Promedio),
    Precio_Final = last(Precio_Promedio),
    Variacion_Porcentual = (Precio_Final - Precio_Inicial) / Precio_Inicial * 100
  )

# Mostrar resultados
variacion %>% arrange(desc(Variacion_Porcentual))

```

```

## # A tibble: 1 x 4
##   Producto          Precio_Inicial Precio_Final Variacion_Porcentual
##   <chr>              <dbl>         <dbl>         <dbl>
## 1 POLLO ENTERO LIMPIO      6.62          5.7         -13.9

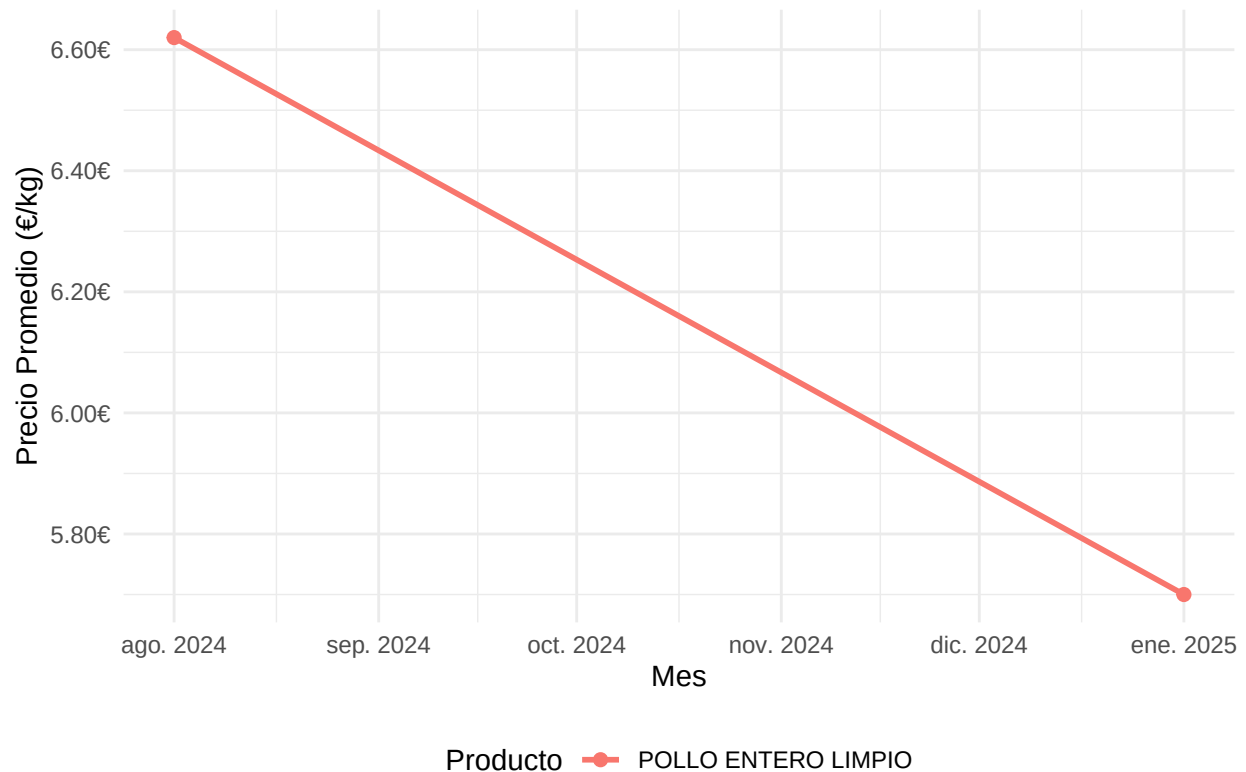
```

```

# Crear gráfico de evolución
ggplot(precio_promedio, aes(x = mes, y = Precio_Promedio, color = Producto)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  labs(
    title = "Evolución del Precio de Productos Cárnicos en el Último Año",
    x = "Mes",
    y = "Precio Promedio (€/kg)",
    color = "Producto"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom") +
  scale_y_continuous(labels = scales::dollar_format(prefix = "", suffix = "€")) +
  scale_x_date(date_labels = "%b %Y", date_breaks = "1 month")

```

## Evolución del Precio de Productos Cárnicos en el Último Año



- ¿Qué marca de lácteos es la más económica en Mercadona?

```
# Filtrar productos de leche
productos_leche <- c("LECHE", "LÁCTEO", "YOGUR", "CREMA", "LECHE ENTERA", "LECHE SEMI-ENTERA", "YOGURT")

productos_de_leche <- productos_global %>%
  filter(str_detect(toupper(Descripcion), paste(productos_leche, collapse = "|"))) %>%
  mutate(
    Marca = str_extract(Descripcion, "[A-Za-zÁÉÍÓÚÑáéíóúñ\\s]+") # Extraer la marca del nombre del producto
  )

# Agrupar por marca y producto, luego calcular el precio máximo, mínimo y medio
leche_por_marca <- productos_de_leche %>%
  group_by(Marca, Descripcion) %>%
  summarise(
    Precio_Maximo = max(P.Unit, na.rm = TRUE), # Precio más alto del producto
    Precio_Minimo = min(P.Unit, na.rm = TRUE), # Precio más bajo del producto
    Precio_Medio = mean(c(Precio_Maximo, Precio_Minimo), na.rm = TRUE), # Precio medio entre el máximo y mínimo
    .groups = "drop"
  ) %>%
  group_by(Marca) %>%
  summarise(
    Precio_Maximo_Marca = max(Precio_Maximo, na.rm = TRUE), # Máximo precio de cualquier producto de la marca
    Precio_Minimo_Marca = min(Precio_Minimo, na.rm = TRUE), # Mínimo precio de cualquier producto de la marca
    Precio_Medio_Marca = mean(Precio_Medio, na.rm = TRUE), # Precio medio entre los productos de la marca
    .groups = "drop"
  ) %>%
```

```

arrange(Precio_Medio_Marca) # Ordenamos por el precio medio de cada marca

# Mostrar la tabla resultante
leche_por_marca

## # A tibble: 53 x 4
##   Marca Precio_Maximo_Marca Precio_Minimo_Marca Precio_Medio_Marca
##   <chr> <dbl> <dbl> <dbl>
## 1 "PASTEL CREMA " 0.4 0.4 0.4
## 2 "LECHE SEMI ~ 0.83 0.83 0.83
## 3 "LECHE SEMI ~ 0.83 0.83 0.83
## 4 "CAFÉ LECHE CAPPU~ 0.85 0.85 0.85
## 5 "LECHE SEMI ~ 0.85 0.85 0.85
## 6 "YOGUR FRESA" 1 0.76 0.88
## 7 "LECHE ENTERA ~ 0.94 0.91 0.925
## 8 "LECHE ENTERA ~ 0.94 0.94 0.94
## 9 "YOGUR COCO ~ 0.95 0.95 0.95
## 10 "YOGUR COCO ~ 0.95 0.95 0.95
## # i 43 more rows

```

- ¿Cuál es el chocolate más comprado por los consumidores?

```

# Primero identificamos los productos de chocolate en la lista
productos_chocolate <- productos_global %>%
  filter(str_detect(toupper(Descripcion),
    "CHOCO|CHOCOLATE|CACAO|COBERTURA|TABLETA")) %>%
  mutate(Descripcion = str_trim(Descripcion))

# Ahora contamos las compras por tipo de chocolate
chocolates_populares <- productos_chocolate %>%
  group_by(Producto = Descripcion) %>%
  summarise(
    Veces_Comprado = n(),
    Cantidad_Total = sum(as.numeric(Peso), na.rm = TRUE),
    Precio_Promedio = mean(as.numeric(P.Unit), na.rm = TRUE)
  ) %>%
  arrange(desc(Veces_Comprado))

# Mostrar los 10 chocolates más comprados
head(chocolates_populares, 10)

```

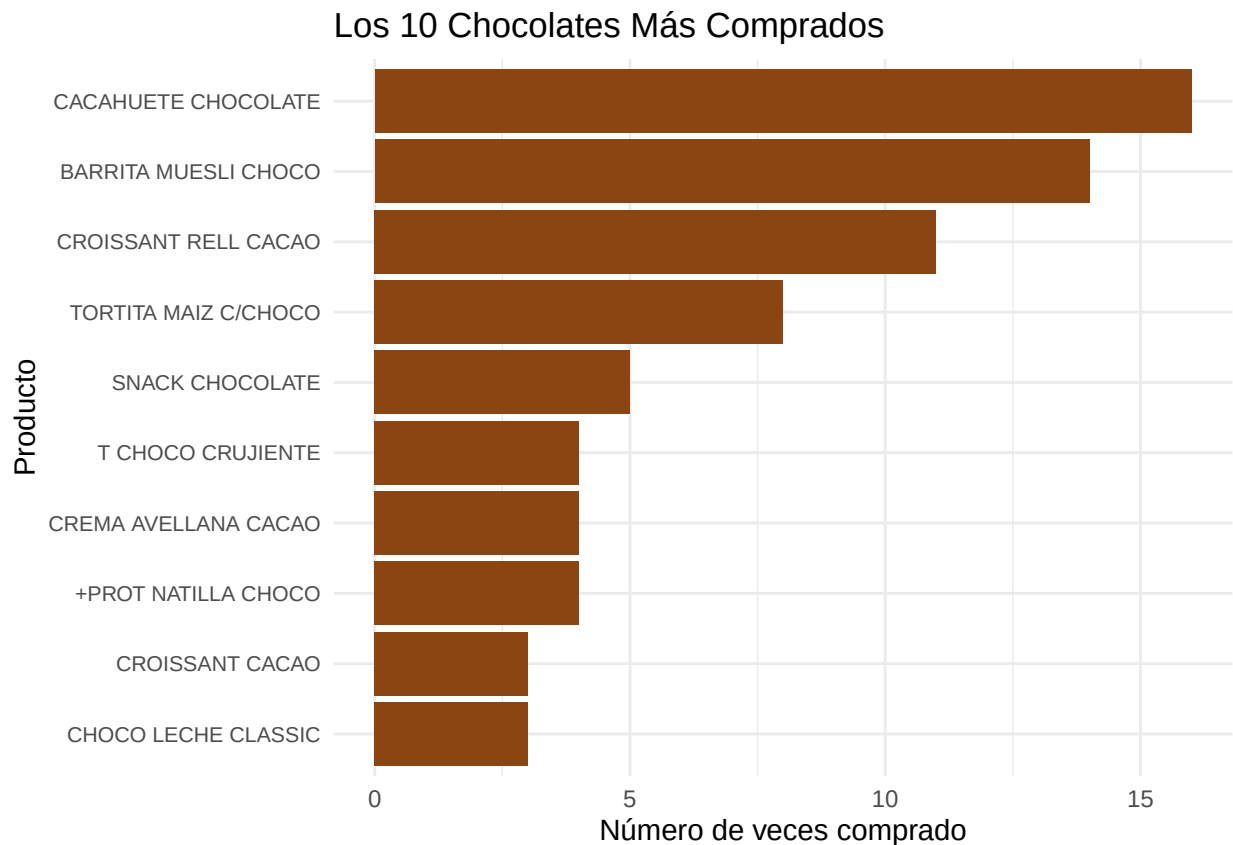
```

## # A tibble: 10 x 4
##   Producto Veces_Comprado Cantidad_Total Precio_Promedio
##   <chr> <int> <dbl> <dbl>
## 1 CACAHUETE CHOCOLATE 16 0 2.28
## 2 BARRITA MUESLI CHOCO 14 0 1.53
## 3 CROISSANT RELL CACAO 11 0 1.90
## 4 TORTITA MAIZ C/CHOCO 8 0 1.73
## 5 SNACK CHOCOLATE 5 0 1.39
## 6 +PROT NATILLA CHOCO 4 0 1.75
## 7 CREMA AVELLANA CACAO 4 0 2.4
## 8 T CHOCO CRUJIENTE 4 0 1.6

```

```
## 9 CHOCO LECHE CLASSIC          3          0          1.05
## 10 CROISSANT CACAO             3          0          1.7
```

```
# Gráfico de barras de los chocolates más comprados
ggplot(head(chocolates_populares, 10),
  aes(x = reorder(Producto, Veces_Comprado), y = Veces_Comprado)) +
  geom_bar(stat = "identity", fill = "#8B4513") +
  coord_flip() +
  labs(
    title = "Los 10 Chocolates Más Comprados",
    x = "Producto",
    y = "Número de veces comprado"
  ) +
  theme_minimal() +
  theme(axis.text.y = element_text(size = 8))
```



- ¿Qué tipo de carne (ternera, cerdo, pollo o pavo) ofrece mejor relación calidad-precio según la frecuencia de compra y el precio promedio?

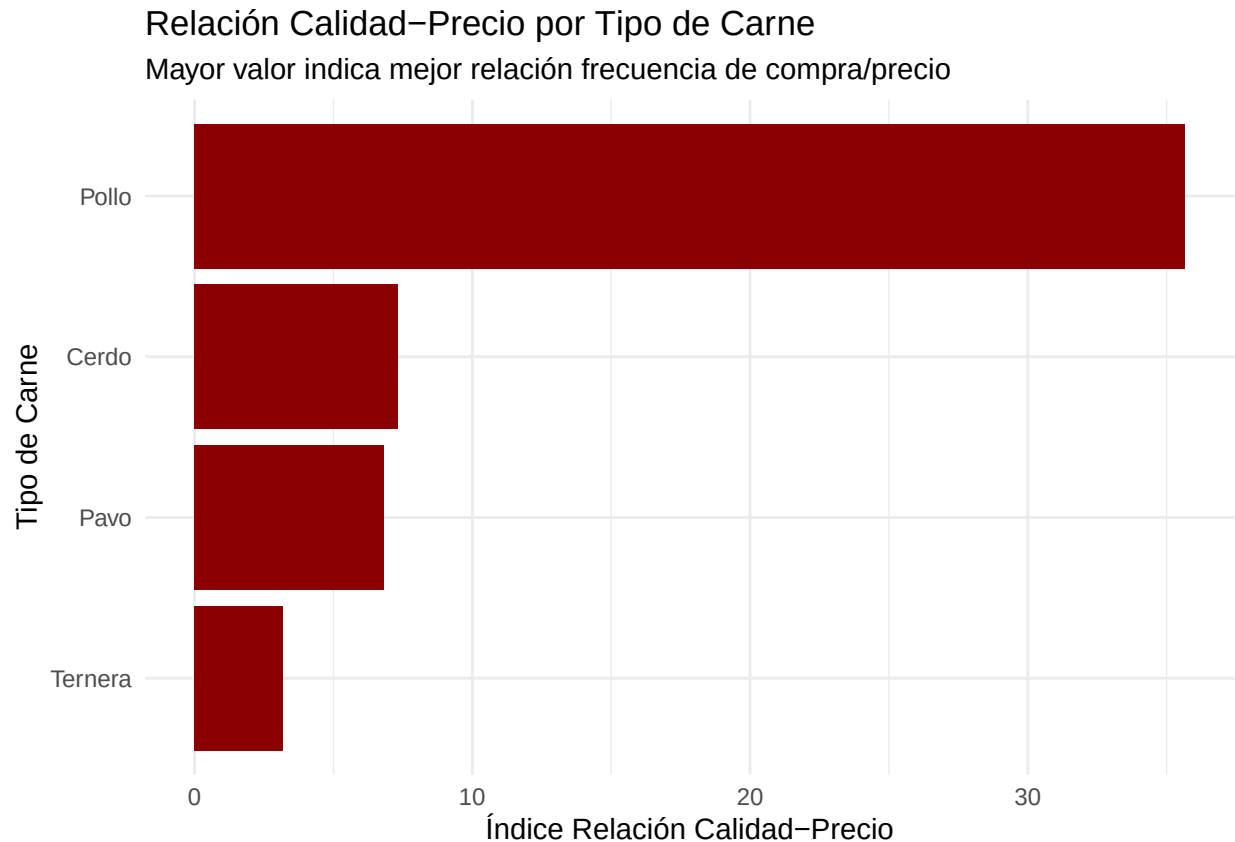
```
# Análisis de relación calidad-precio por tipo de carne
relacion_calidad_precio <- productos_global %>%
  # Filtrar productos cárnicos principales
  filter(str_detect(toupper(Descripcion), "TERNERA|CERDO|POLLO|PAVO|VACUNO")) %>%
  mutate(
    # Clasificar por tipo de carne
    tipo_carne = case_when(
```

```

    str_detect(toupper(Descripcion), "TERNERA|VACUNO") ~ "Ternera",
    str_detect(toupper(Descripcion), "CERDO") ~ "Cerdo",
    str_detect(toupper(Descripcion), "POLLO") ~ "Pollo",
    str_detect(toupper(Descripcion), "PAVO") ~ "Pavo",
    TRUE ~ "Otro"
  ),
  P.Unit = as.numeric(P.Unit)
) %>%
group_by(tipo_carne) %>%
summarise(
  Precio_Promedio = mean(P.Unit, na.rm = TRUE),
  Frecuencia_Compra = n(),
  Relacion_Calidad_Precio = Frecuencia_Compra / Precio_Promedio
) %>%
arrange(desc(Reunion_Calidad_Precio))

# Visualización
ggplot(reunion_calidad_precio, aes(x = reorder(tipo_carne, Relacion_Calidad_Precio), y = Relacion_Calidad_Precio)) +
  geom_bar(stat = "identity", fill = "#8B0000") +
  labs(title = "Relación Calidad-Precio por Tipo de Carne",
        subtitle = "Mayor valor indica mejor relación frecuencia de compra/precio",
        x = "Tipo de Carne",
        y = "Índice Relación Calidad-Precio") +
  theme_minimal() +
  coord_flip()

```



- ¿Qué productos han subido más de precio tras los cambios en el IVA?

```
# Crear tabla de productos únicos (1 registro por producto)
productos_unicos <- productos_global %>%
  group_by(Descripcion) %>%
  summarise(
    P.Unit = first(P.Unit), # Tomamos el primer precio encontrado para cada producto
    .groups = 'drop'
  )

# Calcular precios con y sin IVA para todos los productos únicos
resultados_completos <- productos_unicos %>%
  mutate(
    IVA_aplicado = 21, # Asignamos 21% a todos
    Precio_sin_IVA = round(P.Unit / 1.21, 2),
    Diferencia_IVA = round(P.Unit - Precio_sin_IVA, 2)
  ) %>%
  arrange(desc(Diferencia_IVA)) %>%
  select(
    Producto = Descripcion,
    Precio_con_IVA = P.Unit,
    Precio_sin_IVA,
    Diferencia_IVA,
  )

# Mostrar todos los productos ordenados por mayor diferencia de IVA
resultados_completos
```

```
## # A tibble: 1,586 x 4
##   Producto                                Precio_con_IVA Precio_sin_IVA Diferencia_IVA
##   <chr>                                <dbl>         <dbl>         <dbl>
## 1 "ALISTADO MEDIANO"                    31.3          25.9          5.44
## 2 "QUISQUILLA MEDIANA"                  27.9          23.1          4.84
## 3 "LENGUADO FRESC EUROPEO"              25.0          20.6          4.33
## 4 "QUESO AÑEJO FUERTE"                  19.2          15.9          3.34
## 5 "MEDIO SALMÓN"                        17.1          14.1          2.96
## 6 "BOGAVANTE CRUDO"                     14.8          12.3          2.58
## 7 "BOGAVANTE CRUDO"                     14.8          12.3          2.58
## 8 "ENTRECOT"                            12.9          10.7          2.24
## 9 "RAPE FRESCO ENTERO"                  12.9          10.7          2.24
## 10 "LOTE SOPLO INTENSE"                  12.5          10.3          2.17
## # i 1,576 more rows
```

- ¿Cómo varían los precios de frutas y verduras según la temporada?

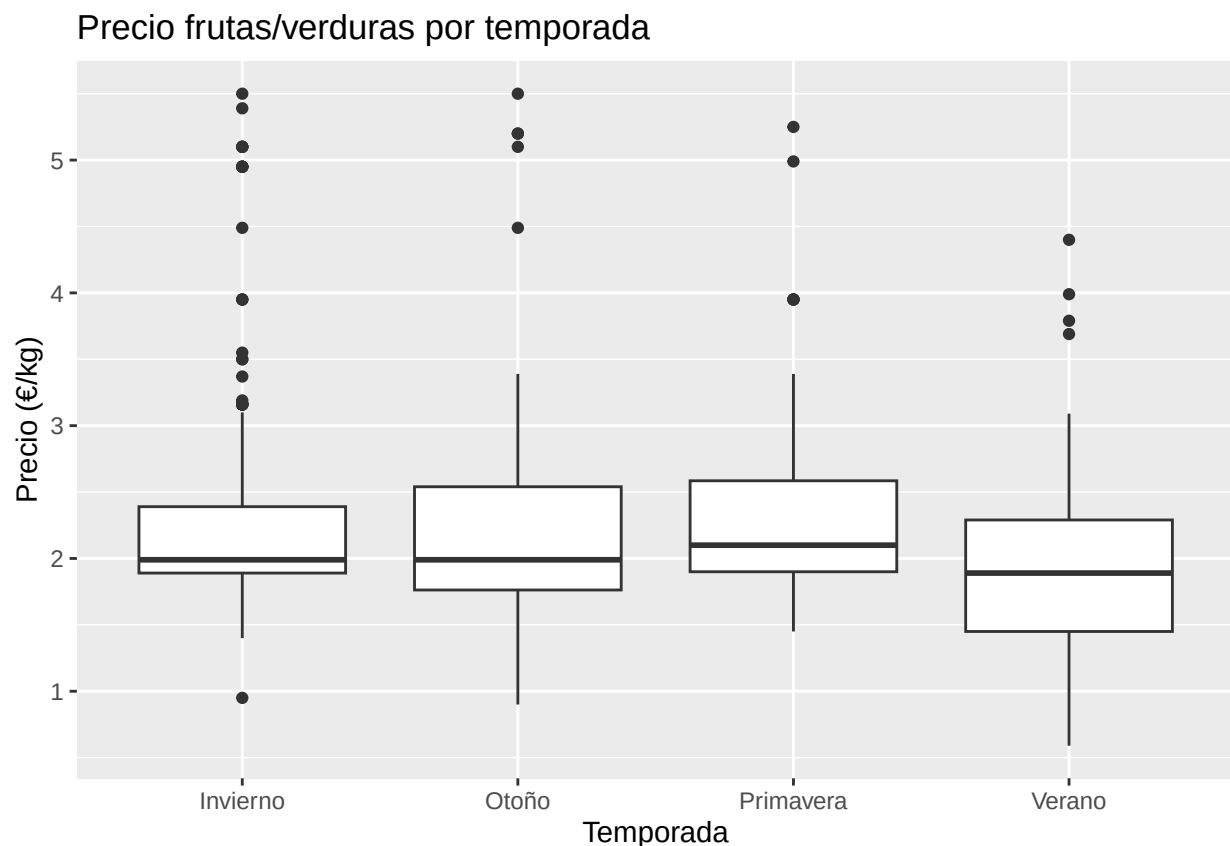
```
datos_completos <- productos_global %>%
  left_join(tickets_global %>% select(factura_simplificada, fecha),
    by = "factura_simplificada")
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many" to silence this warning.
```

```

datos_completos %>%
  filter(Tipo_peso == "Verdura y Frutas") %>%
  mutate(mes = month(dmy(fecha)),
         temporada = case_when(
           mes %in% 3:5 ~ "Primavera",
           mes %in% 6:8 ~ "Verano",
           mes %in% 9:11 ~ "Otoño",
           TRUE ~ "Invierno"
         )) %>%
  ggplot(aes(x = temporada, y = P.Unit)) +
  geom_boxplot() +
  labs(title = "Precio frutas/verduras por temporada",
       x = "Temporada",
       y = "Precio (€/kg)")

```



- ¿Qué categoría de productos representa el mayor gasto por ticket?

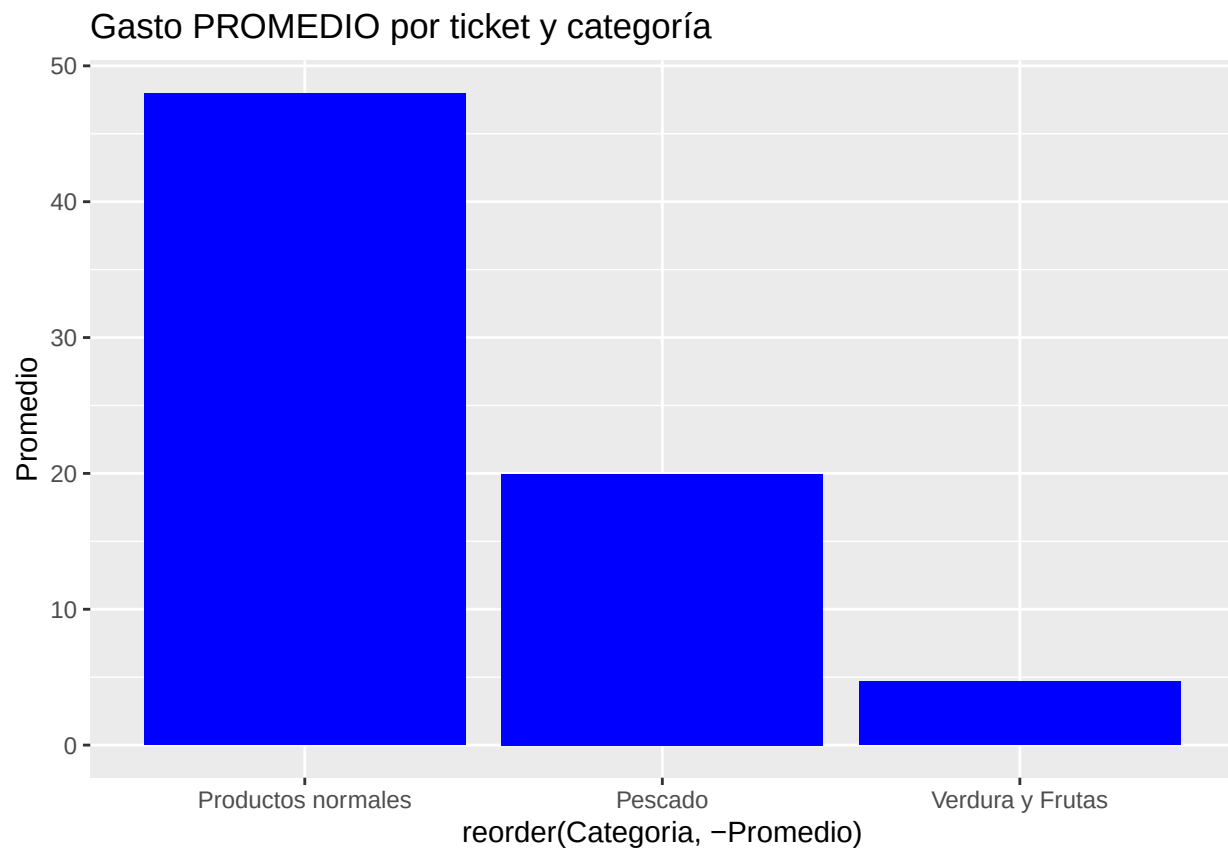
```

productos_global %>%
  left_join(tickets_global %>% select(factura_simplificada), by = "factura_simplificada") %>%
  mutate(
    Categoria = ifelse(is.na(Tipo_peso), "Productos normales", Tipo_peso),
    Importe = ifelse(is.na(Importe), P.Unit, Importe)
  ) %>%
  group_by(factura_simplificada, Categoria) %>%
  summarise(Gasto_categoria = sum(Importe), .groups = "drop") %>%

```

```
group_by(Categoria) %>%
  summarise(Promedio = mean(Gasto_categoria)) %>% # Promedio del gasto por ticket
  ggplot(aes(reorder(Categoria, -Promedio), Promedio)) +
  geom_col(fill = "blue") +
  labs(title = "Gasto PROMEDIO por ticket y categoría")
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada), : Detected an unexpected many-to-many relationship between 'x' and 'y'.
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
##   "many-to-many"' to silence this warning.
```



- ¿Qué marca de café prefieren los consumidores?

```
cafes <- productos_global %>%
  filter(str_detect(tolower(Descripcion), "café")) %>%
  count(Descripcion, sort = TRUE)

cafes
```

```
##           Descripcion n
## 1  CAFÉ NATURAL HOGAR 2
## 2  CAFÉ SOLUBLE CLASSIC 2
## 3  CARAMELO CAFÉ S/AZUC 2
```



```
## 4      +PROTEÍNAS CAFÉ 1
## 5 CAFÉ LECHE CAPPUCCIN 1
```

- ¿El precio de un mismo producto cambia dependiendo del día en que se compra?

```
productos_global2 <- productos_global %>%
  left_join(tickets_global %>% select(factura_simplificada, fecha),
            by = "factura_simplificada")
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many"' to silence this warning.
```

```
# Buscamos productos con más de un precio distinto
productos_con_precios_distintos <- productos_global2 %>%
  filter(!is.na(P.Unit), !is.na(Descripcion)) %>%
  group_by(Descripcion) %>%
  summarise(num_precios_distintos = n_distinct(P.Unit)) %>%
  filter(num_precios_distintos > 1)

productos_con_precios_distintos
```

```
## # A tibble: 445 x 2
##   Descripcion          num_precios_distintos
##   <chr>                  <int>
## 1 + PROTEÍNA ARANDANOS             2
## 2 1/2 CONEJO TROCEADO            12
## 3 1/2 POLLO PART CERT             4
## 4 100 % TRIGO ESPELTA             2
## 5 12 HUEVOS CAMPEROS              4
## 6 12 HUEVOS GRANDES-L            4
## 7 12 HUEVOS SUPER GRAN           4
## 8 200 SERVIL. BLANCAS             2
## 9 3 VEGETALES                    2
## 10 4 ESTACIONES                   2
## # i 435 more rows
```

- ¿Cuántos productos por ticket se compran en promedio?

```
productos_por_ticket <- productos_global %>%
  group_by(factura_simplificada) %>%
  tally()

mean(productos_por_ticket$n)
```

```
## [1] 19.55477
```

- ¿Cuál es el día de la semana que Mercadona factura más?

```

# Debemos de ver en que día de la semana se .
# Hay que convertir la fecha en días de la semana y el importe total
# en numerico cambiando la coma por el punto

tickets <- tickets_global %>% mutate(fecha = as.Date(fecha, format = "%d/%m/%Y"),
                                   dia_semana = weekdays(fecha),
                                   importe_total = as.numeric(gsub(",", ".", importe_total))) %>%
  select(dia_semana, importe_total) %>% # Cogemos unicamente estas dos columnas, ya que las otras no
  group_by(dia_semana) %>% # Agrupamos por días de la semana para calcular el importe de cada día.
  summarise(total_dia = sum(importe_total, na.rm = TRUE)) %>% # Sumatorio que suma todos los importes
  arrange(desc(total_dia)) # Ordenamos de mayor a menor el data.frame

cat("El día de la semana donde Mercadona a recaudado más es:", tickets$dia_semana[1])

## El día de la semana donde Mercadona a recaudado más es: sábado

# Si queremos ver la tabla de valores:
print(tickets)

```

```

## # A tibble: 7 x 2
##   dia_semana total_dia
##   <chr>         <dbl>
## 1 sábado         4517.
## 2 lunes          2499.
## 3 viernes        2024.
## 4 martes         1739.
## 5 miércoles      1667.
## 6 jueves          983.
## 7 domingo         75.0

```

- ¿En qué estaciones del año se consume más pescado?

```

# Tenemos que calcular el mes del ticket y incluirlo en su estación correspondiente:

meses <- tickets_global %>%
  mutate(mes = as.Date(fecha, format = "%d/%m/%Y"), # Convertimos en formato fecha la fecha.
         mes = month(mes)) %>% # De fecha a mes.
  select(mes, factura_simplificada) %>% # Seleccionamos solamente las columnas que necesitamos.
  mutate(estacion = ifelse(mes %in% c(12, 1, 2), "Invierno", # Columna estaciones con condiciones para s
                           ifelse(mes %in% c(3:5), "Primavera",
                                   ifelse(mes %in% c(6:8), "Verano", "Otoño")))) %>%
  group_by(estacion) %>% # Agrupamos por estaciones
  select(estacion, factura_simplificada) # Seleccionamos las columnas que nos interesan

estacion_pescado <- inner_join(meses, productos_global, by = "factura_simplificada") %>%
  filter(!is.na(Importe)) %>%
  filter(Tipo_peso == "Pescado") %>%
  group_by(estacion) %>%
  summarise(total_pescado = n()) %>%
  arrange(desc(total_pescado))

```

```

## Warning in inner_join(meses, productos_global, by = "factura_simplificada"): Detected an unexpected

```

```
## i Row 1 of 'x' matches multiple rows in 'y'.
## i Row 96 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many" to silence this warning.
```

```
print(estacion_pescado)
```

```
## # A tibble: 4 x 2
##   estacion total_pescado
##   <chr>         <int>
## 1 Invierno         29
## 2 Primavera        18
## 3 Otoño            2
## 4 Verano           1
```

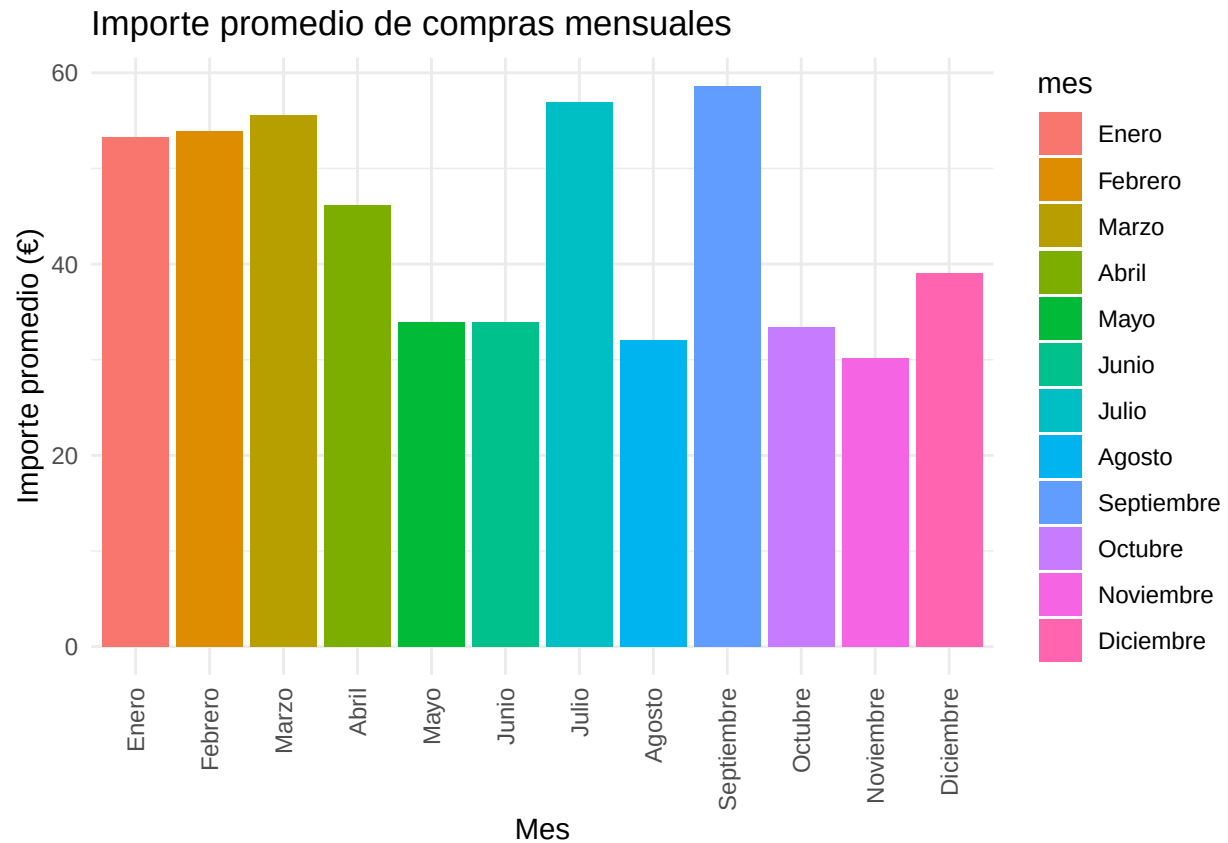
*#Como podemos observar las estaciones donde mas se consume pescado son invierno y primavera.*

- ¿Cuál es el importe total promedio de una compra mensual dependiendo del mes?

```
# Utilizaremos el data frame "meses" con algunos cambios.
# Transformación del data frame con cálculo de importe promedio mensual
meses <- tickets_global %>%
  mutate(fecha = as.Date(fecha, format = "%d/%m/%Y"), # Convertimos la fecha a formato Date
         mes = month(fecha), # Extraemos el número del mes
         importe_total_num = as.numeric(gsub(",", ".", importe_total))) %>% # Convertimos el importe a
  group_by(mes) %>%
  summarise(media_importe = mean(importe_total_num, na.rm = TRUE)) # Calculamos el promedio por mes

# Convertimos la columna mes a factor con niveles explícitos para que ggplot ordene correctamente
meses <- meses %>%
  mutate(mes = factor(mes, levels = 1:12, labels = c("Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio",
                                                    "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre")))

# Generamos el gráfico de barras con ggplot
ggplot(meses, aes(x = mes, y = media_importe, fill = mes)) +
  geom_bar(stat = "identity") + # Se usa 'stat = "identity"' para indicar que los valores de Y ya están en el data frame
  labs(title = "Importe promedio de compras mensuales",
       x = "Mes",
       y = "Importe promedio (€)") +
  theme_minimal() + # Se usa un tema limpio y moderno
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) # Rotamos los nombres de los meses
```



```
print(meses) # Mostramos la tabla con los valores exactos.
```

```
## # A tibble: 12 x 2
##   mes      media_importe
##   <fct>      <dbl>
## 1 Septiembre    58.6
## 2 Julio         56.9
## 3 Marzo        55.6
## 4 Febrero      53.9
## 5 Enero        53.3
## 6 Abril        46.1
## 7 Diciembre    39.0
## 8 Junio        33.9
## 9 Mayo         33.9
## 10 Octubre     33.4
## 11 Agosto      32.0
## 12 Noviembre   30.2
```

- ¿Cuáles son los productos que Mercadona podría quitar por su baja demanda?

```
# Calculamos la frecuencia de cada producto en el conjunto de datos
productos_frecuencia <- productos_global %>%
  group_by(Descripcion) %>%
  summarise(frecuencia = n()) %>%
  arrange(frecuencia)
```

```
# Seleccionamos los productos con menor frecuencia (baja demanda)
productos_baja_demanda <- productos_frecuencia %>%
  filter(frecuencia <= quantile(frecuencia, 0.25)) # Tomamos el 25% menos comprado

# Mostramos los productos con baja demanda
print(productos_baja_demanda)
```

```
## # A tibble: 542 x 2
##   Descripcion      frecuencia
##   <chr>          <int>
## 1 +PROT PLA-AÇAÍ          1
## 2 +PROTEÍNAS CAFÉ          1
## 3 1/2 POLLO PARTIDO          1
## 4 12 HUEVOS MEDIANOS -          1
## 5 12 HUEVOS MEDIANOS-M          1
## 6 18 HUEVOS CODORNIZ          1
## 7 30 B.ENVASES C.FÁCIL          1
## 8 6 HUEVOS COCIDOS          1
## 9 6 HUEVOS GRANDES-L          1
## 10 A. ALIÑADA PICADEDOS          1
## # i 532 more rows
```

```
# Todos estos productos se han comprado solamente una vez en total.
```

- ¿Cual es el pescado y hortaliza (verdura o fruta) más caro?

```
# Filtramos los productos de pescado y ordenamos por precio unitario descendente
productos_pescado_mas_carro <- productos_global %>%
  filter(!is.na(Tipo_peso)) %>% # Eliminamos valores NA en Tipo_peso
  filter(Tipo_peso == "Pescado") %>% # Filtramos solo productos que sean pescado
  arrange(desc(P.Unit)) %>% # Ordenamos por precio unitario de mayor a menor
  select(Descripcion, P.Unit) # Seleccionamos solo las columnas relevantes
```

```
# Mostramos el pescado más caro
head(productos_pescado_mas_carro, 1) # Extraemos solo el producto más caro
```

```
##           Descripcion P.Unit
## 1 QUISQUILLA MEDIANA 27.9
```

```
# Filtramos los productos de verduras y frutas (hortalizas)
productos_verdura_mas_carro <- productos_global %>%
  filter(!is.na(Tipo_peso)) %>% # Eliminamos valores NA en Tipo_peso
  filter(Tipo_peso != "Pescado") %>% # Filtramos los productos que NO sean pescado
  arrange(desc(P.Unit)) %>% # Ordenamos por precio unitario de mayor a menor
  select(Descripcion, P.Unit) # Seleccionamos solo las columnas relevantes
```

```
# Mostramos la verdura/fruta más cara
head(productos_verdura_mas_carro, 1) # Extraemos solo el producto más caro
```

```
##           Descripcion P.Unit
## 1      AGUACATE      5.5
```

```
# El pescado más caro es la quisquilla mediana con un precio de 27,9 €/kg
# La verdura/fruta más cara es el aguacate con un precio de 5,5 €/kg
```

- ¿Qué productos tienen mayor carga impositiva por IVA?

```
# NOTA: En este contexto, "tipos_iva" se encuentra en el data frame 'ticket_global'.
# Por ello, unimos 'productos_global' con 'ticket_global' usando la columna 'factura_simplificada'
# para incorporar la información del IVA a cada producto.
```

```
productos_iva <- productos_global %>%
  left_join(tickets_global %>% select(factura_simplificada, tipos_iva),
            by = "factura_simplificada") %>%
  filter(!is.na(Importe), !is.na(tipos_iva)) %>%
  mutate(
    # Extraemos el mayor valor de IVA del campo "tipos_iva"
    # Se usa str_extract_all para conseguir todos los porcentajes y luego se calcula el máximo.
    max_iva = tipos_iva %>%
      str_extract_all("\\d+%") %>%
      sapply(function(x) max(as.numeric(str_remove(x, "%")))),
    # Convertimos el campo "Importe" a número, reemplazando la coma decimal por punto
    Importe = as.numeric(str_replace(Importe, ",", "."))
  )
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many" to silence this warning.
```

```
# Agrupamos por "Descripcion" y "max_iva", sumamos el importe total en cada grupo
# y ordenamos, primero, por el IVA en forma descendente y luego por el importe total.
```

```
productos_por_iva <- productos_iva %>%
  group_by(Descripcion, max_iva) %>%
  summarise(Total_con_IVA = sum(Importe, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(max_iva), desc(Total_con_IVA))
```

```
# Mostramos el resultado final: los productos con mayor carga impositiva por IVA.
productos_por_iva
```

```
## # A tibble: 89 x 3
##   Descripcion      max_iva Total_con_IVA
##   <chr>          <dbl>      <dbl>
## 1 PLATANO         21         94.4
## 2 SALMON ENTERO   21         77.6
## 3 SEPIA LONJA     21         65.7
## 4 HABAS           21         56.8
## 5 BACALADILLA     21         40.6
## 6 BANANA          21         39.6
## 7 RODABALLO       21         38.6
## 8 SEPIA FRESCA    21         32.1
## 9 CALABACIN VERDE 21          29
## 10 SEPIA SUCIA REFRIG 21         28.7
## # i 79 more rows
```

- ¿Qué tipo de pan se compra con más frecuencia?

```
# Filtramos solo productos que contengan "PAN" en su descripción (ignorando mayúsculas/minúsculas)
pan_mas_frecuente <- productos_global %>%
  filter(grepl("PAN", Descripcion, ignore.case = TRUE)) %>% # Filtra productos relacionados con pan
  count(Descripcion, sort = TRUE) %>% # Cuenta cada tipo de pan
  slice_head(n = 1) # Selecciona el más frecuente

pan_mas_frecuente
```

```
##      Descripcion  n
## 1 PAN DE PUEBLO 25
```

- ¿Los consumidores compran más productos frescos o envasados?

```
# Contamos los productos frescos y envasados según la descripción
productos_comprados <- productos_global %>%
  mutate(tipo_producto = case_when(
    grepl("fresco", Descripcion, ignore.case = TRUE) ~ "Fresco",
    grepl("envasado", Descripcion, ignore.case = TRUE) ~ "Envasado",
    TRUE ~ "Otro"
  )) %>%
  count(tipo_producto) %>% # Contamos cada tipo
  filter(tipo_producto != "Otro") # Filtramos los casos que no sean Fresco ni Envasado

productos_comprados
```

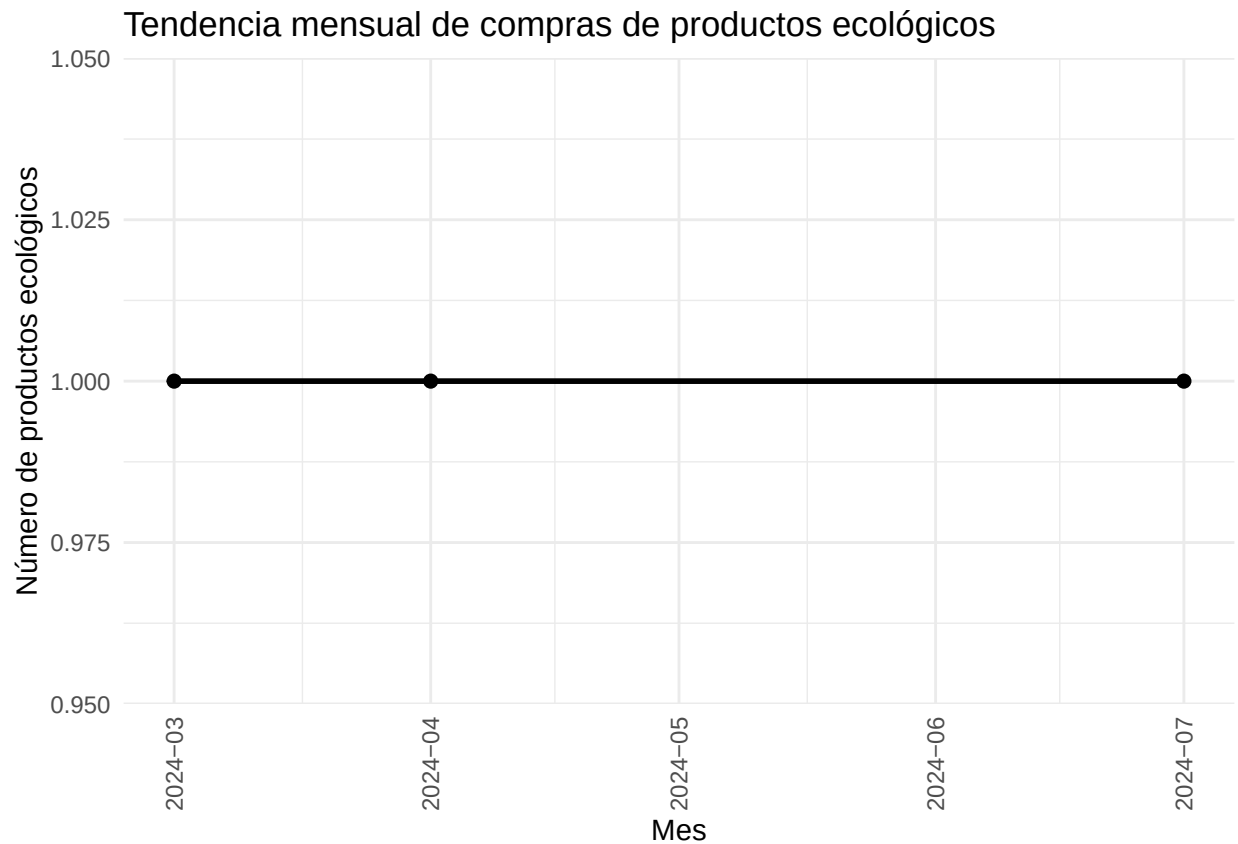
```
##      tipo_producto  n
## 1      Envasado    2
## 2      Fresco    42
```

- ¿Se observa una tendencia de aumento o disminución en la compra de productos ecológicos?

```
# 1) Filtramos productos ecológicos (ó/acento y organico/orgánico)
productos_ecologicos <- productos_global %>%
  filter(
    grepl("ecol[óo]gico|bio|org[áa]nico", Descripcion, ignore.case = TRUE)
  ) %>%
# 2) Unimos sólo la fecha del ticket (ajusta 'factura_simplificada' si tu key es otra)
left_join(
  tickets_global %>% select(factura_simplificada, fecha),
  by = "factura_simplificada"
) %>%
filter(!is.na(fecha)) %>%
# 3) Parsiando seguro de fecha
mutate(fecha = dmy(fecha),
       fecha_anno_mes = format(fecha, "%Y-%m"))

# 4) Contamos por mes
tendencia_ecologico <- productos_ecologicos %>%
  count(fecha_anno_mes) %>%
  arrange(fecha_anno_mes)
```

```
# 5) Si hay datos, gráfico:
if(nrow(tendencia_ecologico) > 0) {
  ggplot(tendencia_ecologico,
    aes(x = as.Date(paste0(fecha_anno_mes, "-01")), y = n)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  scale_x_date(date_breaks = "1 month", date_labels = "%Y-%m") +
  labs(
    title = "Tendencia mensual de compras de productos ecológicos",
    x = "Mes",
    y = "Número de productos ecológicos"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, vjust = 0.5))
} else {
  message("No se han encontrado productos ecológicos con esas claves en Descripcion.")
}
```



- ¿Qué porcentaje del total del ticket corresponde al IVA en promedio?

```
# Calculamos, para cada ticket, el porcentaje de IVA que representa parte del total.
ticket_con_iva <- tickets_global %>%
  mutate(
    # Convertimos el importe total de carácter a número, reemplazando la coma decimal por punto.
```



```

importe_total_num = as.numeric(gsub(",", ".", importe_total)),

# Extraemos todos los porcentajes de IVA (ej. "10%", "21%") y nos quedamos con el valor máximo,
# en caso de existir varios en un mismo ticket.
max_iva = sapply(str_extract_all(tipos_iva, "\\d+%"), function(x) {
  # Convertimos cada valor (eliminando el símbolo "%") a número y seleccionamos el máximo.
  max(as.numeric(str_remove(x, "%")), na.rm = TRUE)
}),

# Calculamos el porcentaje del total que corresponde al IVA.
# Si un ticket tiene tasa R, entonces, el IVA incluido en el total es R/(100+R)*100.
porcentaje_iva = (max_iva / (100 + max_iva)) * 100
)

# Calculamos el porcentaje promedio de IVA para todos los tickets.
porcentaje_iva_promedio <- ticket_con_iva %>%
  summarise(porcentaje_promedio = mean(porcentaje_iva, na.rm = TRUE))

# Mostramos el resultado
porcentaje_iva_promedio

##   porcentaje_promedio
## 1                14.54102

```

- ¿Hay productos que se compran sistemáticamente juntos (patrones de compra)?

```

# Generar pares de productos comprados juntos
pares_lista <- productos_global %>%
  group_by(factura_simplificada) %>%
  summarise(productos = list(unique(Descripcion)), .groups = "drop") %>%
  filter(lengths(productos) >= 2) %>% # Filtrar tickets con al menos 2 productos
  pull(productos) # Extraer solo la lista de productos

# Generar todas las combinaciones de productos por factura
pares <- unlist(
  lapply(pares_lista, function(x) {
    combn(x, 2, FUN = function(par) paste(sort(par), collapse = " + "), simplify = TRUE)
  }),
  use.names = FALSE
)

# Contar frecuencia de cada par
pares_frecuentes <- as.data.frame(table(pares)) %>%
  arrange(desc(Freq))

# Mostrar los pares más frecuentes
head(pares_frecuentes, 10)

```

```

##               pares Freq
## 1  ALMENDRA NATURAL + CACAHUETE CHOCOLATE  16
## 2                ALMENDRA NATURAL + PEPINO  16
## 3      CACAHUETE CHOCOLATE + PEPINO  16

```

```
## 4      ALMENDRA NATURAL + BARRITA MUESLI CHOCO    14
## 5      BARRITA MUESLI CHOCO + CACAHUETE CHOCOLATE  14
## 6              BARRITA MUESLI CHOCO + PEPINO     14
## 7              PEPINO + Z.NARANJA 330ML REF.     14
## 8      ALMENDRA NATURAL + Z.NARANJA 330ML REF.   13
## 9      CACAHUETE CHOCOLATE + Z.NARANJA 330ML REF. 13
## 10             FILETE PECHUGA + PLATANO          13
```

- ¿Cuánto gasta en promedio una persona en productos de limpieza por ticket?

```
# Filtramos productos de limpieza por palabras clave
productos_limpieza <- productos_global %>%
  filter(str_detect(tolower(Descripcion), "fregar|bayeta|multiusos|escurridor|limpieza|agua mineral|fibra"))

# Sumamos el gasto por ticket para esos productos
gasto_limpieza_por_ticket <- productos_limpieza %>%
  group_by(factura_simplificada) %>%
  summarise(gasto_total = sum(as.numeric(P.Unit), na.rm = TRUE))

# Calculamos el gasto promedio
gasto_promedio_limpieza <- mean(gasto_limpieza_por_ticket$gasto_total)

print(paste("Gasto promedio en productos de limpieza por ticket:", round(gasto_promedio_limpieza, 2)))
```

```
## [1] "Gasto promedio en productos de limpieza por ticket: 3.26"
```

- ¿Qué producto ha tenido más variación de precio a lo largo del tiempo?

```
# Unimos productos con sus fechas y filtramos datos válidos
productos_con_fecha <- productos_global %>%
  left_join(
    tickets_global %>% select(factura_simplificada, fecha),
    by = "factura_simplificada"
  ) %>%
  filter(!is.na(P.Unit)) %>%
  mutate(fecha = as.Date(fecha, format = "%d/%m/%Y"))
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many-to-many relationship.
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship = "many-to-many"' to silence this warning.
```

```
# Calculamos el rango de precios por producto, solo si aparece al menos 3 veces
variacion_precios <- productos_con_fecha %>%
  group_by(Descripcion) %>%
  filter(n() >= 3) %>% # Al menos 3 apariciones
  summarise(
    rango = max(P.Unit, na.rm = TRUE) - min(P.Unit, na.rm = TRUE)
  ) %>%
  arrange(desc(rango)) %>%
  head(10)
```

```
# Mostramos los productos con mayor variación de precio
print("Productos con mayor variación de precio (rango):")
```

```
## [1] "Productos con mayor variación de precio (rango):"
```

```
print(variacion_precios)
```

```
## # A tibble: 10 x 2
##   Descripcion      rango
##   <chr>          <dbl>
## 1 LECHE ENTERA P6    7.19
## 2 LENGUADO FRESC EUROP 6
## 3 LECHE DESNAT. CALCIO 4.87
## 4 LECHE DESN S/LACT 4.7
## 5 ESCALOPIN SALMÓN 4.51
## 6 ACEITE VIRGEN     4.25
## 7 LANGOSTINO COCIDO 4.22
## 8 ACEITE VIRGEN EX 4.05
## 9 JAMON CEBO IBERICO 3.91
## 10 LECHE SEMI P6    3.54
```

- ¿Cuáles son los productos más comprados en total y cuál es su gasto acumulado?

```
# Contar cuántas veces se compró cada producto y cuánto se ha gastado en total en cada uno
productos_global %>%
  group_by(Descripcion) %>%
  summarise(
    veces_comprado = n(),
    gasto_total = sum(P.Unit, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(veces_comprado)) %>%
  head(20) # Mostrar los 20 más comprados
```

```
## # A tibble: 20 x 3
##   Descripcion      veces_comprado gasto_total
##   <chr>          <int>      <dbl>
## 1 "PLATANO"             58      122.
## 2 "ATUN CLARO OLIVA"   53      84.8
## 3 "QUESO LONCHAS CABRA" 49      141.
## 4 "FILETE PECHUGA"    34      145.
## 5 "UVA BLANCA S/SEM"   33      87.7
## 6 "BOLSA PLASTICO"    32       4.8
## 7 "PARKING"           32       0
## 8 "12 HUEVOS GRANDES-L" 25      57.4
## 9 "BANANA"            25      36.2
## 10 "PAN DE PUEBLO"     25      38.9
## 11 "PEPINO"           24      42.9
## 12 "ZANAHORIA BOLSA"  24      25.8
## 13 "AGUACATE BANDEJA" 22      74.3
## 14 "12 HUEVOS SUPER GRAN" 21      70.7
```

|                              |    |      |
|------------------------------|----|------|
| ## 15 "ATÚN CLARO OLIVA PK6" | 21 | 110. |
| ## 16 "CANÓNICOS"            | 21 | 23.9 |
| ## 17 "FUET ESPETEC EXTRA"   | 21 | 54.0 |
| ## 18 "TOM. RECETA ARTESANA" | 21 | 45.2 |
| ## 19 "BERENJENA"            | 20 | 36.9 |
| ## 20 "KIWI VERDE"           | 20 | 65.5 |

- ¿Se puede detectar algún cambio en los hábitos de consumo antes y después de periodos vacacionales?

```
# Añadir fechas a los productos y clasificar cada compra según su periodo (pre-vacacional, vacacional,
gasto_por_periodo <- productos_global %>%
  left_join(tickets_global %>% select(factura_simplificada, fecha), by = "factura_simplificada") %>%
  mutate(fecha = as.Date(fecha, format = "%d/%m/%Y")) %>%
  mutate(tipo_periodo = case_when(
    fecha >= as.Date("2023-12-05") & fecha < as.Date("2023-12-20") ~ "Pre-vacacional",
    fecha >= as.Date("2023-12-20") & fecha <= as.Date("2024-01-07") ~ "Vacacional",
    fecha > as.Date("2024-01-07") & fecha <= as.Date("2024-01-22") ~ "Post-vacacional",
    TRUE ~ "Normal"
  )) %>%

# Calcular el gasto total por ticket y clasificarlo por tipo de periodo
group_by(factura_simplificada, tipo_periodo) %>%
  summarise(total_ticket = sum(P.Unit, na.rm = TRUE), .groups = "drop") %>%
# Calcular el gasto promedio por tipo de periodo
group_by(tipo_periodo) %>%
  summarise(
    gasto_promedio = mean(total_ticket, na.rm = TRUE),
    tickets = n()
  ) %>%
  arrange(desc(gasto_promedio))
```

```
## Warning in left_join(., tickets_global %>% select(factura_simplificada, : Detected an unexpected many
## i Row 96 of 'x' matches multiple rows in 'y'.
## i Row 1 of 'y' matches multiple rows in 'x'.
## i If a many-to-many relationship is expected, set 'relationship =
## "many-to-many"' to silence this warning.
```

```
print(gasto_por_periodo)
```

```
## # A tibble: 4 x 3
##   tipo_periodo      gasto_promedio tickets
##   <chr>          <dbl>     <int>
## 1 Post-vacacional    151.         9
## 2 Vacacional        114.        10
## 3 Pre-vacacional     80.5         4
## 4 Normal            46.7       260
```

## Conclusión.

A partir del análisis realizado, podemos concluir que el algoritmo desarrollado en R es capaz de extraer y estructurar de manera eficaz la información contenida en los tickets de compra en PDF de Mercadona. Esto ha permitido observar patrones relevantes en el comportamiento de los precios y el gasto.

En particular, se ha identificado una variación significativa en los precios de ciertos productos a lo largo del tiempo, lo que podría estar relacionado con ajustes en el IVA y otros factores económicos. Además, se observó que el impacto del IVA no siempre se transfiere de forma uniforme al precio final, sugiriendo estrategias comerciales o subsidios específicos por producto.

También se logró determinar los periodos con mayor gasto promedio por ticket, lo cual podría asociarse a fechas especiales o promociones. Finalmente, la automatización de este proceso representa una herramienta útil para estudios de consumo, facilitando análisis longitudinales que serían inviables de realizar manualmente.